

You're welcome! Now, let's dive into **React Router** and cover all the essential concepts with a **proper folder structure** and **examples**.

What is React Router?

React Router is a **library** that helps in handling **navigation (routing)** in a React application.

◇ Why Use React Router?

- ✓ **Single Page Application (SPA)** - Allows navigation **without full-page reloads**.
- ✓ **Dynamic Routing** - Routes change dynamically based on the URL.
- ✓ **Easy Navigation** - Provides functions like `useNavigate()` for programmatic navigation.
- ✓ **Route Parameters** - Extract dynamic values from the URL using `useParams()`.
- ✓ **Protected Routes** - Restrict access to certain pages based on authentication.

Folder Structure for React Router

```
/react-router-app
├── /src
│   ├── /components
│   │   ├── Home.jsx
│   │   ├── About.jsx
│   │   ├── Contact.jsx
│   │   ├── Dashboard.jsx
│   │   ├── Login.jsx
│   │   ├── UserProfile.jsx
│   │   └── ProtectedRoute.jsx
│   ├── /pages
│   │   └── NotFound.jsx
│   ├── /App.js
│   └── /index.js
├── /package.json
├── /public
└── ...
```

Step 1: Install React Router

First, install `react-router-dom` in your React project:

```
npm install react-router-dom
```

Step 2: Setup BrowserRouter in index.js

🔗 The **BrowserRouter** component wraps the entire application to enable routing.

📌 **index.js**

```
import React from "react";
import ReactDOM from "react-dom";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Step 3: Define Routes in App.js

🔗 The **Routes** component contains all our **Route** components, which define **path-to-component mapping**.

📌 **App.js**

```
import { Routes, Route } from "react-router-dom";
import Home from "./components/Home";
import About from "./components/About";
import Contact from "./components/Contact";
import Dashboard from "./components/Dashboard";
import Login from "./components/Login";
import UserProfile from "./components/UserProfile";
import NotFound from "./pages/NotFound";
import ProtectedRoute from "./components/ProtectedRoute";

const App = () => {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />

      {/* Protected Route */}
      <Route
        path="/dashboard"
        element={
          <ProtectedRoute>
```

```
        <Dashboard />
      </ProtectedRoute>
    }
  />

  { /* Dynamic Route with useParams */}
  <Route path="/user/:id" element={<UserProfile />} />

  { /* 404 Page */}
  <Route path="*" element={<NotFound />} />
</Routes>
);
};

export default App;
```

Step 4: Create Component Files

Now, let's create all the components used in `App.js`.

1 Home.jsx

👉 Basic home page

```
import { Link } from "react-router-dom";

const Home = () => {
  return (
    <div>
      <h1>Home Page</h1>
      <nav>
        <Link to="/about">About</Link> | <Link to="/contact">Contact</Link>
      </nav>
    </div>
  );
};

export default Home;
```

2 About.jsx

```
const About = () => {
  return <h1>About Page</h1>;
};
```

```
export default About;
```

3 Contact.jsx

```
const Contact = () => {  
  return <h1>Contact Page</h1>;  
};  
  
export default Contact;
```

4 Dashboard.jsx (Protected Route)

```
const Dashboard = () => {  
  return <h1>Welcome to the Dashboard (Protected Page)</h1>;  
};  
  
export default Dashboard;
```

5 Login.jsx

```
import { useNavigate } from "react-router-dom";  
  
const Login = () => {  
  const navigate = useNavigate();  
  
  const handleLogin = () => {  
    localStorage.setItem("auth", "true");  
    navigate("/dashboard");  
  };  
  
  return (  
    <div>  
      <h1>Login Page</h1>  
      <button onClick={handleLogin}>Login</button>  
    </div>  
  );  
};  
  
export default Login;
```

6 UserProfile.jsx (Using useParams)

🔗 We use `useParams()` to **extract user ID from the URL**.

```
import { useParams } from "react-router-dom";

const UserProfile = () => {
  const { id } = useParams();

  return <h1>User Profile: {id}</h1>;
};

export default UserProfile;
```

☑ URL Example:

If the user visits `/user/101`, they will see:

```
User Profile: 101
```

7 ProtectedRoute.jsx (Restricting Access)

🔗 If the user is not logged in, they are redirected to the login page.

```
import { Navigate } from "react-router-dom";

const ProtectedRoute = ({ children }) => {
  const isAuthenticated = localStorage.getItem("auth") === "true";

  return isAuthenticated ? children : <Navigate to="/login" />;
};

export default ProtectedRoute;
```

8 NotFound.jsx (404 Page)

🔗 **Handles invalid routes**

```
const NotFound = () => {
  return <h1>404 - Page Not Found</h1>;
};

export default NotFound;
```

React Router Hooks

1 `useNavigate()` - Programmatic Navigation

🔑 Used to navigate dynamically (instead of `<Link>`).

```
import { useNavigate } from "react-router-dom";

const GoBackButton = () => {
  const navigate = useNavigate();

  return <button onClick={() => navigate(-1)}>Go Back</button>;
};

export default GoBackButton;
```

- ☑ `navigate(-1)`: Moves to the **previous page**
- ☑ `navigate("/dashboard")`: Redirects to `/dashboard`

2 `useLocation()` - Get Current URL

🔑 Fetches the current URL details

```
import { useLocation } from "react-router-dom";

const LocationExample = () => {
  const location = useLocation();

  return <h1>Current Path: {location.pathname}</h1>;
};

export default LocationExample;
```

- ☑ If the user is on `/contact`, it will display:

```
Current Path: /contact
```

Summary

Concept	Description
---------	-------------

Concept	Description
BrowserRouter	Wraps the entire application to enable routing
Routes	Contains all Route components
Route	Defines the path-to-component mapping
useNavigate()	Allows programmatic navigation
useParams()	Extracts dynamic values from the URL
useLocation()	Gets the current route information
Protected Routes	Restricts access based on authentication

Next Steps

Want to build a **React Router project with authentication, API calls, and nested routes**? Let me know! 

